

Startup Ecosystem Analytics Platform

Architecture & DevOps Review Presentation

Presenter: DevOps & Principal Solutions Architect

Deployment Stack: FastAPI · Neo4j 5.13 · Redis 7 · Nginx · Jenkins

Project Vision – Platform Objective

Platform Objective

Create a technical intelligence platform mapping founders, startups, and investor relationships.

- ▶ **Graph Topology:** Models natural networks without joining relational tables.
- ▶ **Safe Capital Flow:** Enables programmatic funding commitments on wallets.

Primary Goals

- ▶ **Query Latency:** Replace slow recursive SQL JOIN operations.
- ▶ **Concurrency:** Eliminate double-allocation conflicts on investment rounds.
- ▶ **Throughput:** Deliver search matching recommendations in sub-15ms.

Project Vision – Portal Roles

Startups & Investors

- ▶ **Startups:** Manage profile attributes, request round commitments, and showcase verified milestones.
- ▶ **Investors:** Track preferred sectors, manage transaction wallets, and receive match leads.

Analysts & Admin Users

- ▶ **Analysts:** Oversee database health, approve profile registrations, and verify achievements.
- ▶ **System Admin:** Audit wallets, track system health, and clear stale cache entries.

Problem Statement – Relational Pitfalls

SQL Database Bottleneck

- ▶ Relational tables require heavy JOINS to map multi-level connections.
- ▶ Computing match scores across 3rd-degree networks causes database locks.
- ▶ Performance degrades exponentially as nodes increase.

Lack of Transaction Protection

- ▶ Concurrent investment requests to the same round cause oversubscription.
- ▶ Race conditions lead to wallet balances drifting out of sync.
- ▶ Manual interventions needed for database correction.

Problem Statement – Workflow Obstacles

Unverified Achievement Lifecycles

- ▶ Founders self-report milestone metrics without validation.
- ▶ Inability to trace, audit, or verify historical round data points.
- ▶ Leads to investor distrust and inaccurate leaderboard states.

API Latency & Blocking Threads

- ▶ Heavy matchmaking computations block the server main request loop.
- ▶ Slow analytical lookups cause UI components to freeze or timeout.
- ▶ Limits platform scalability under concurrent user loads.

Proposed Solution – Graph Database (Neo4j)

Native Graph Storage Schema

- ▶ Maps entities (Founders, Investors, Startups) as nodes.
- ▶ Models actions (invested, interested) as physical graph edges.
- ▶ Eliminates the mismatch between application models and storage.

Fast Structural Traversals

- ▶ Evaluates path matching queries in microseconds.
- ▶ Computes network similarity scores without SQL JOIN overheads.
- ▶ Leverages index-free adjacency for relationship walks.

Proposed Solution – Memory Cache (Redis)

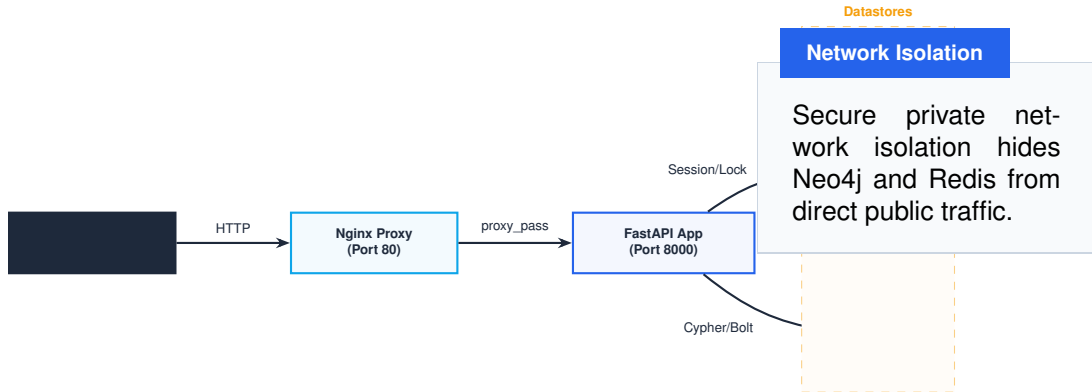
Caching & Result Set Buffering

- ▶ Caches heavy query results indexed by MD5 parameter hashes.
- ▶ Automatically invalidates profiles on data mutations.
- ▶ Serves read-heavy client requests in under 2ms.

Distributed State Synchronization

- ▶ Uses Redis lock leases to synchronize simultaneous transfer requests.
- ▶ Sorted Sets keep live ranking leaderboards updated.
- ▶ Tracks active sessions and coordinates distributed resources.

High-Level System Architecture



System Architecture – Gateway Layer (Nginx)

Traffic Routing Configuration

- ▶ Exposes a single public IP mapping paths on Port 80.
- ▶ Reverse-proxies dynamic routes downstream to Python web-workers.
- ▶ Secures APIs by filtering unexpected request protocols.

Static Resource Caching

- ▶ Serves static asset files directly from host volumes.
- ▶ Decreases backend request load by bypassing Python runtime.
- ▶ Configured with gzip compression to minimize page load times.

System Architecture – Async Controller (FastAPI)

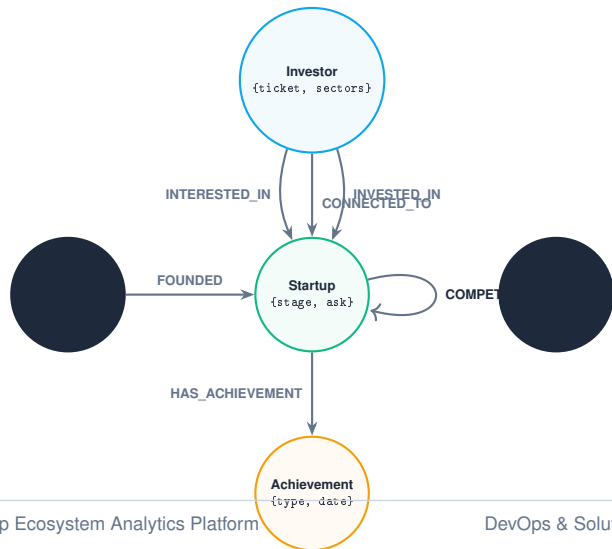
Asynchronous Route Operations

- ▶ Runs routes using Python's async-await concurrency framework.
- ▶ Implements dependency injection for connection management.
- ▶ Uses Pydantic to validate input JSON schemas strictly.

Connection Pool Management

- ▶ Connects to graph databases using persistent connection pools.
- ▶ Lifespan handlers bootstrap datastores and close drivers cleanly.
- ▶ Recycles dead database sessions automatically.

Database Schema – Graph Entity Model



Graph Properties

Modeling relationships as native entities allows the database to store property fields directly on edges.

Database Schema – Edge Relationship Properties

Attributes on Graph Edges

- ▶ Edges store crucial fields directly (e.g. `amount` on `INVESTED_IN`).
- ▶ Eliminates auxiliary lookup tables for relational data.
- ▶ Keeps historical transaction metadata close to nodes.

Structural Network Proximity

- ▶ Captures competitive relationships via `COMPETES_WITH` loops.
- ▶ Enables tracing of co-investment networks through shared nodes.
- ▶ Allows sector path traversals without index lookups.

Database Schema – Entity Constraints

Database Integrity Constraints

- ▶ Uniqueness rules configured on Startup IDs and Investor IDs.
- ▶ Prevents identity duplicates at the database server level.
- ▶ Restricts orphan relationships from corrupting the graph.

Automatic Execution Script

- ▶ Constraints created programmatically at API startup.
- ▶ Validates database schema structures during container bootstrap.
- ▶ Logs error states if constraints fail validation.

Database Schema – Query Indexing

Fast Property Lookup Indexes

- ▶ Secondary indices created on startup sector and funding stages.
- ▶ Speeds up match filters by avoiding full graph scans.
- ▶ Indexes investor preferred locations dynamically.

Performance Improvements

- ▶ Node property indexing reduces lookup times to sub-millisecond rates.
- ▶ Allows the engine to scale efficiently to thousands of nodes.
- ▶ Dynamic query planning benefits from updated indexing stats.

Core Engine – Hybrid Matchmaking Algorithm

Sector & Capital Alignment

- ▶ **Sector Match (40 pts):** Sector lists overlapping investor interests.
- ▶ **Ticket Overlap (30 pts):** Funding ask falls within investor ticket limits.
- ▶ **Stage Focus (20 pts):** Capital rounds align with investor focus stage.

Network & Velocity Enhancements

- ▶ **Network Bump (10 pts):** Co-investment paths are within two hops.
- ▶ **Milestone Velocity (10 pts):** Quick achievement velocity (≥ 3 milestones in 90 days).
- ▶ **Total Score:** Max score is 110 points.

Matchmaking – Cypher Query Structure

Match Scoring Execution

```
MATCH (i:Investor), (s:Startup {id: $startup_id})
WITH i, s,
  CASE WHEN s.sector IN i.preferred_sectors THEN 40
  ELSE 0 END AS sector_score,
  CASE WHEN i.ticket_min <= s.funding_ask
    AND s.funding_ask <= i.ticket_max THEN 30
  ELSE 0 END AS ticket_score,
  CASE WHEN s.stage IN i.stage_focus THEN 20
  ELSE 0 END AS stage_score
OPTIONAL MATCH path = (i)-[:INVESTED_IN*1..2]->(other:Startup)
WHERE other.sector = s.sector
WITH i, s, sector_score, ticket_score, stage_score,
  CASE WHEN count(path) > 0 THEN 10 ELSE 0 END AS network_score
OPTIONAL MATCH (s)-[:HAS_ACHIEVEMENT]->(a:Achievement)
WHERE a.date >= date() - duration({days: 90})
WITH i, sector_score, ticket_score, stage_score,
  network_score,
  CASE WHEN count(a) >= 3 THEN 10 ELSE 0 END AS ach_score
RETURN i.id, (sector_score + ticket_score + stage_score +
  network_score + ach_score) AS total_score
ORDER BY total_score DESC LIMIT 10
```

Execution Profile

- ▶ Logic executes inside the Neo4j query planner.
- ▶ Minimizes runtime CPU load on Python web workers.
- ▶ Traverses up to 2-hop investor relations in microseconds.

Matchmaking – Cypher Execution Details

Cypher Case Optimization

- ▶ Computes logical scores entirely in the database engine memory.
- ▶ Minimizes CPU usage on the FastAPI web-app worker nodes.
- ▶ Uses pre-compiled query templates for speed.

Dynamic Path Traversals

- ▶ Traverses co-invested paths (`INVESTED_IN*1..2`) recursively.
- ▶ Caps returned matched sets to minimize serialization overhead.
- ▶ Avoids memory pagination issues with database `LIMIT` clauses.

Concurrency Control – Double-Spend Vulnerabilities

The Oversubscription Threat

- ▶ High-frequency investor transfers can exceed a startup's round limit.
- ▶ Simultaneous writes read stale balances before committing values.
- ▶ Leads to over-allocation of funding equity.

Relational Isolation Gaps

- ▶ Graph transaction scopes alone do not lock transient state across threads.
- ▶ Distributed locks are required to serialize transfer actions.
- ▶ Avoids deadlocks by applying timeouts to connection queues.

Concurrency Control – Two-Tier Protection

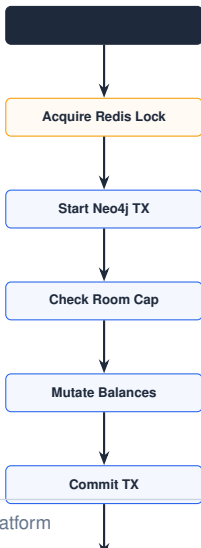
Distributed Locking (Redis)

- ▶ Backend acquires a lock on the startup ID before checking balances.
- ▶ Sets a 10s auto-expiry lease to prevent deadlocks on worker failures.
- ▶ Uses redlock principles to handle distributed clusters.

ACID Graph Transactions

- ▶ Transfers mutate balances and update graphs inside one session.
- ▶ Rejects transaction changes if the startup round limit is breached.
- ▶ Rollback triggers automatically on database constraint errors.

Transaction Lifecycle – Wallet Flow Sequence



Operational Flow

Synchronizes parallel transactions to ensure round data limits are preserved.

High-Performance Cache – Result Buffering

Query Hashing (MD5)

- ▶ Generates unique keys by hashing query parameters.
- ▶ Caches responses under `feed:filtered:{hash}` for 120 seconds.
- ▶ Limits CPU-intensive recommendations on repeated searches.

Active Invalidation Flow

- ▶ Any profile update triggers dynamic cache clearing.
- ▶ Keeps search data consistent without long wait cycles.
- ▶ Evicts dependent queries via atomic Redis keys deletes.

High-Performance Cache – Analytics & Leaderboards

Investor Rankings (ZSET)

- ▶ Redis Sorted Sets track scores on `leaderboard:investors`.
- ▶ Increments scores instantly on successful transaction commitments.
- ▶ Returns top ranked lists to dashboards with zero query latency.

Profile Views Tracking

- ▶ Increments click counters using Hash fields (`views:startup:{id}`).
- ▶ Periodic background tasks sync clicks back to the database.
- ▶ Mitigates database write loads from high-traffic profiles.

Security Architecture – Token Verification

Cryptographic JWT Engine

- ▶ Signatures validated using HMAC-SHA256 and custom secrets.
- ▶ Password structures secured with salted bcrypt hashing hashes.
- ▶ Enforces strict lifetime expiration validation on token payloads.

Token Revocation Lists

- ▶ Caches blacklisted tokens in Redis memory during logout actions.
- ▶ Checks incoming IDs against the blacklist during validation.
- ▶ Automatically evicts tokens using Redis TTL settings.

Security Architecture – Access Controls (RBAC)

Decorator-Based Authorization

- ▶ Restricts specific routes using custom decorators.
- ▶ Rejects request handling if user role claims are invalid.
- ▶ Validates scope hierarchies dynamically per request thread.

NoSQL Injection Protections

- ▶ Parametrizes all Cypher queries at connection execution.
- ▶ Rejects unescaped input characters during JSON parsing.
- ▶ Limits graph traversal query complexity at driver configuration.

DevOps Automation – Continuous Integration

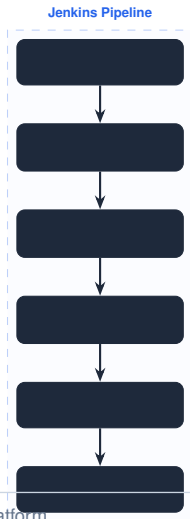
Lint & Syntax Verifications

- ▶ Automatically checks code syntax and formatting rules on PRs.
- ▶ Validates module package dependency structures.
- ▶ Uses auto-formatters to preserve consistent styling rules.

Pytest Verification Automation

- ▶ Runs the test suite in isolation using GitHub Actions.
- ▶ Validates backend routing and database driver connectivity.
- ▶ Rejects changes failing unit verification checks.

DevOps Automation – CD Deployment Pipeline



Continuous Delivery

Pipelines execute automated canary checks during deployments. Rollbacks run instantly if REST health signals timeout.

DevOps Automation – Container Orchestration

Container Isolation Strategy

- ▶ **Frontend Container:** Lightweight Alpine Nginx serves static resources directly.
- ▶ **Backend Container:** Python-slim execution environment hosting FastAPI.
- ▶ **Neo4j Container:** Persistent data folders mapped to host volumes.

Network Isolation Policies

- ▶ Locks backend containers inside private bridge networks.
- ▶ Only Nginx exposes ports to the external public network interfaces.
- ▶ Redis and Neo4j block direct external ingress ports.

System Verification – Pytest Infrastructure

Mock Testing Configurations

- ▶ Simulates graph drivers to test route logics in memory.
- ▶ Restricts external queries to ensure rapid execution speeds.
- ▶ Mocks JWT payloads to bypass login stages during endpoint checks.

Boundary Testing Cases

- ▶ Validates transaction locks under simulated parallel conditions.
- ▶ Asserts that unauthorized role access yields 403 HTTP codes.
- ▶ Tests limit compliance when wallet values edge close to zero.

System Reliability & Health Diagnostics

Active Diagnostics Probes

- ▶ Exposes a unified health route verifying external systems.
- ▶ Returns service connectivity flags and database latency.
- ▶ Enables standard container health checks dynamically.

Error Handling & Retries

- ▶ Middleware records request processing times and errors.
- ▶ Database drivers execute exponential backoff retry loops on fail.
- ▶ Re-establishes broken network connections without application restarts.

Future Roadmap & Project Summary

Technical Growth Roadmap

- ▶ Package container configurations as Kubernetes Helm charts.
- ▶ Implement central nodes search using Graph Data Science.
- ▶ Geo-replicate Redis caches to minimize global read latencies.

Architecture Conclusion

- ▶ **FastAPI Async Core:** High-concurrency operations.
- ▶ **Neo4j Graphs:** Relationships modeled natively inside graph stores.
- ▶ **Redis Buffers:** Sub-15ms search matching suggestions.